

Code Optimization & Profiling

Numba JIT Compilation & Data Type Optimization

26th of February 2026

Jimmy Jessen Nielsen
jjn@es.aau.dk

Dept. of Electronic Systems
Aalborg University





Code Optimization & Profiling

Today's Agenda

- ▶ **Recap & Theory:** Optimization mindset + Algorithmic intensity (35 min)
- ▶ **MP1 briefing:** requirements overview (5 min)
- ▶ **Studio Session:** Self-paced milestones
 - ▶ Milestone 1: Function-level profiling with cProfile
 - ▶ Milestone 2: Line-level profiling with line_profiler
 - ▶ Milestone 3: Implement Numba @njit and benchmark
 - ▶ Milestone 4: Data type optimization (float32 vs float64)
 - ▶ Milestone 5 (extension): Parallel Numba with prange

Work at your own pace. Done early? Polish your MP1 mini-report, push to Git, and you are free to go.

Code Optimization & Profiling

Pre-Lecture Check



Quick poll - raise your hand:

- ▶ Who installed Numba successfully?

Code Optimization & Profiling

Pre-Lecture Check



Quick poll - raise your hand:

- ▶ Who installed Numba successfully?
- ▶ Who has working NumPy Mandelbrot from L2?

Code Optimization & Profiling

Pre-Lecture Check



Quick poll - raise your hand:

- ▶ Who installed Numba successfully?
- ▶ Who has working NumPy Mandelbrot from L2?
- ▶ Who read the Numba documentation?

Code Optimization & Profiling

Pre-Lecture Check



Quick poll - raise your hand:

- ▶ Who installed Numba successfully?
- ▶ Who has working NumPy Mandelbrot from L2?
- ▶ Who read the Numba documentation?
- ▶ What speedup did you achieve with NumPy? (Call out numbers!)

Typical NumPy speedup: 5–50 $\times$ over naive Python



Code Optimization & Profiling

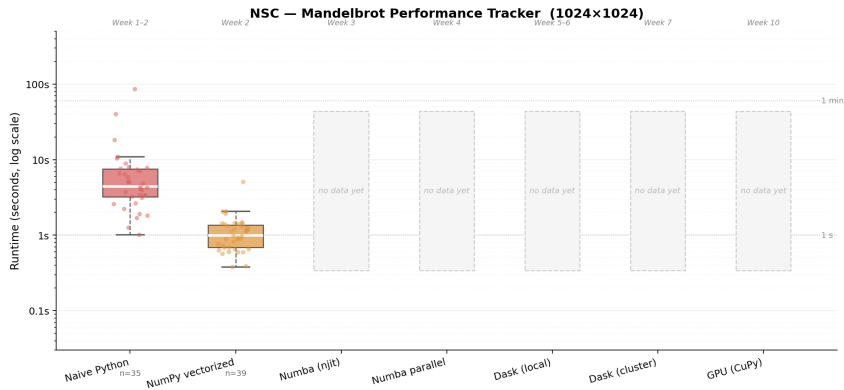
Learning Objectives

- ▶ Understand and calculate **algorithmic intensity** (FLOPs/byte)
- ▶ Distinguish **compute-bound vs. memory-bound** code — and choose the right fix
- ▶ Apply **systematic profiling** to find real bottlenecks
- ▶ Use **Numba @njit** to achieve 50–200× speedup over naive Python
- ▶ Understand **data type tradeoffs** (float32 vs. float64)
- ▶ Produce a **reproducible performance comparison** ready for MP1

By the end of today: three working implementations (Naive, NumPy, Numba), a profiling report, and MP1 essentially done.

How Fast Is Your Mandelbrot?

Class Performance Tracker



1024×1024 grid,. Boxplot (excl. errors >300 s); dots are individual results.

Outliers: slow naive (~86–100 s) are older Intel Core i7 (6th–7th gen); fast NumPy (~0.3 s) is Apple M3.

Updated: February 26, 2026



The Optimization Mindset

Optimization Mindset

Famous Quote & Reality



"Premature optimization is the root of all evil"

– Donald Knuth

What this means:

- ▶ Don't optimize before you know what's slow
- ▶ Don't sacrifice clarity for negligible speedup
- ▶ Don't guess — measure!

What this doesn't mean:

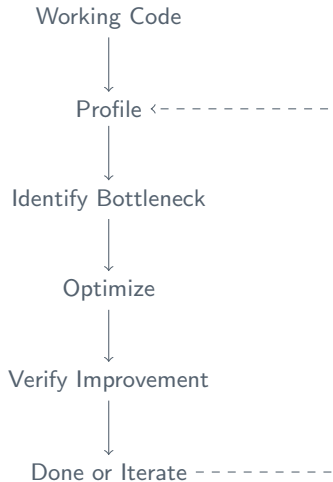
- ▶ Never optimize
- ▶ Ignore performance

The right approach:

1. Make it work (correctness first)
2. Measure performance (profiling)
3. Optimize bottlenecks (data-driven)
4. Verify improvements (measure again)

Optimization Mindset

The Optimization Workflow





Optimization Mindset

Common Optimization Pitfalls

- ▶ **Pitfall 1:** Optimizing the wrong thing
 - ▶ "I'll optimize this function" – but it's only 2% of runtime!
 - ▶ **Solution:** Profile first, optimize highest impact
- ▶ **Pitfall 2:** Making code unreadable
 - ▶ Clever tricks that save 1% but make debugging impossible
 - ▶ **Solution:** Clarity first, then optimize hot paths only
- ▶ **Pitfall 3:** Not measuring improvements
 - ▶ Assume optimization worked without verification
 - ▶ Sometimes "optimizations" actually make things slower!
 - ▶ **Solution:** Always benchmark before and after
- ▶ **Pitfall 4:** Optimizing too early
 - ▶ Spending days optimizing code that runs once per year
 - ▶ **Solution:** Is the code actually slow enough to matter?

Today's mantra: **"Profile first, optimize second, verify always"**



Understanding Performance: Compute vs. Memory

Understanding Performance

Two Types of Bottlenecks

Why do some optimizations work and others don't?

All algorithms are limited by one of two things:

1. **Compute-bound** (CPU speed is the limit)

- ▶ Lots of arithmetic operations per data element
- ▶ Example: Computing $\sin(x)$, $\exp(x)$, matrix multiplication
- ▶ **Optimization:** JIT compilation, GPU acceleration

2. **Memory-bound** (memory bandwidth is the limit)

- ▶ Few operations per data element loaded
- ▶ Example: Summing array, copying data, simple filters
- ▶ **Optimization:** Cache-friendly access, reduce memory traffic

Key insight: Different problems need different optimizations!

Understanding Performance

Algorithmic Intensity

Algorithmic Intensity quantifies if code is compute- or memory-bound:

$$\text{Algorithmic Intensity} = \frac{\text{Number of FLOPs}}{\text{Bytes transferred from memory}}$$

Units: FLOPs/byte (floating-point operations per byte)

Rule of thumb:

- ▶ **Low intensity (< 1 FLOP/byte):** Memory-bound
 - ▶ Cache optimization, memory layout matters most
- ▶ **Medium intensity (1-10 FLOPs/byte):** Balanced
 - ▶ Both CPU and memory matter
- ▶ **High intensity (> 10 FLOPs/byte):** Compute-bound
 - ▶ JIT compilation, GPU acceleration effective

Why this matters: Tells you which optimization strategies will help!

Understanding Performance

Example: Vector Addition

Problem: $\mathbf{c} = \mathbf{a} + \mathbf{b}$ where vectors have N elements

Step 1: Count FLOPs

- ▶ N addition operations
- ▶ **Total FLOPs:** N

Step 2: Count memory transfers

- ▶ Read **a**: N elements $\times$ 8 bytes (float64) = $8N$ bytes
- ▶ Read **b**: N elements $\times$ 8 bytes = $8N$ bytes
- ▶ Write **c**: N elements $\times$ 8 bytes = $8N$ bytes
- ▶ **Total bytes:** $24N$ bytes

Step 3: Calculate intensity

$$\text{Intensity} = \frac{N \text{ FLOPs}}{24N \text{ bytes}} = \frac{1}{24} \approx 0.04 \text{ FLOPs/byte}$$

Conclusion: **Memory-bound!** JIT won't help much. Cache optimization needed.

Understanding Performance

Example: Matrix-Matrix Multiply

Problem: $\mathbf{C} = \mathbf{A} \times \mathbf{B}$ where matrices are $M \times M$

Step 1: Count FLOPs

- ▶ Each element C_{ij} requires M multiplications + M additions
- ▶ For $M \times M$ result: M^2 elements $\times 2M$ operations each
- ▶ **Total FLOPs:** $2M^3$

Step 2: Count memory transfers (naive implementation)

- ▶ Read all of **A**: $M^2 \times 8$ bytes
- ▶ Read all of **B**: $M^2 \times 8$ bytes
- ▶ Write all of **C**: $M^2 \times 8$ bytes
- ▶ **Total bytes:** $24M^2$ bytes (ignoring cache reuse for now)

Step 3: Calculate intensity

$$\text{Intensity} = \frac{2M^3 \text{ FLOPs}}{24M^2 \text{ bytes}} = \frac{M}{12} \text{ FLOPs/byte}$$

Conclusion: For large M (> 100), this is **compute-bound!** GPU/JIT very effective.

Understanding Performance

Example: Mandelbrot Set

Your turn! Let's analyze the Mandelbrot computation:

Inner loop (per pixel):

```
z = 0j
while n < max_iter and z.real**2 + z.imag**2 <= 4.0:
    z = z*z + c; n += 1
```

Step 1: Count FLOPs (per iteration)

- ▶ $z \times z$: 1 complex multiply ≈ 4 real FLOPs¹
- ▶ $+c$: 1 complex addition = 2 real FLOPs
- ▶ $|z|^2$ escape test: 2 squares + 1 add = 3 FLOPs
- ▶ **Total per iteration: ≈ 9 FLOPs; for $\text{max_iter} = 100$: up to ~ 900 FLOPs/pixel**

Step 2: Memory transfers

- ▶ Read c once: 16 bytes (complex128)
- ▶ Write result once: 4 bytes (int32 iteration count)
- ▶ **Total: ≈ 20 bytes per pixel**

¹Formally $4 \text{ mults} + 2 \text{ adds} = 6 \text{ ops}$, but modern CPUs use FMA instructions that fuse multiply-add into one op, giving ≈ 4 effective FLOPs.

Understanding Performance

Mandelbrot Analysis

Step 3: Calculate intensity

$$\text{Intensity} = \frac{900 \text{ FLOPs}}{20 \text{ bytes}} = 45 \text{ FLOPs/byte}$$

Conclusion: Mandelbrot is **highly compute-bound!**

What this tells us:

- ▶ NumPy helps (vectorization reduces Python overhead)
- ▶ **Numba will help A LOT** (removes interpreter, native code)
- ▶ GPU will be extremely effective (parallel compute) — we will revisit this in L10
- ▶ Memory optimization and cache layout not critical (each pixel independent)

Prediction: Numba should give 50–200× speedup (platform-dependent)!

Let's verify this in Milestone 3...



Understanding Performance

Practice: Estimating Algorithmic Intensity

Pairwise discussion activity (10 min):

Discuss with your neighbour and estimate for each algorithm:

- ▶ FLOPs per output element
- ▶ Memory traffic (bytes read + written)
- ▶ Arithmetic intensity = FLOPs / bytes — compute- or memory-bound?

Three algorithms to analyse in the coming slides.

→ **Discuss each of them first, then advance to check your answers.**



Understanding Performance

Practice: Solutions (1/3) — Computing $\sin(x)$

Computing $\sin(x)$ for each element in an array (hint: Taylor series)

²A minimax polynomial is fitted to minimise the maximum error across the full input range, unlike a Taylor series which is only accurate close to the expansion point. NumPy delegates to the platform's `libm` (e.g. Intel SVML), which uses such approximations internally.

Understanding Performance

Practice: Solutions (1/3) — Computing $\sin(x)$

Computing $\sin(x)$ for each element in an array (hint: Taylor series)

FLOPs: ~ 20 per element

- Taylor series: ~ 10 terms, each a multiply-add ≈ 2 FLOPs per term

Memory: 8 bytes read + 8 bytes write = 16 bytes per element

²A minimax polynomial is fitted to minimise the maximum error across the full input range, unlike a Taylor series which is only accurate close to the expansion point. NumPy delegates to the platform's `libm` (e.g. Intel SVML), which uses such approximations internally.

Understanding Performance

Practice: Solutions (1/3) — Computing $\sin(x)$

Computing $\sin(x)$ for each element in an array (hint: Taylor series)

FLOPs: ~ 20 per element

- Taylor series: ~ 10 terms, each a multiply-add ≈ 2 FLOPs per term

Memory: 8 bytes read + 8 bytes write = 16 bytes per element

Arithmetic intensity: $20/16 \approx 1.25$ FLOPs/byte $\rightarrow$ **Balanced / borderline compute-bound**

In practice, NumPy (and the CPU's math library) does *not* use Taylor series — it uses a minimax polynomial approximation² that achieves the same accuracy with fewer operations.

²A minimax polynomial is fitted to minimise the maximum error across the full input range, unlike a Taylor series which is only accurate close to the expansion point. NumPy delegates to the platform's `libm` (e.g. Intel SVML), which uses such approximations internally.

Understanding Performance

Practice: Solutions (2/3) — Array Sum

Summing all elements of an array: $s = \sum_{i=1}^N a_i$

Understanding Performance

Practice: Solutions (2/3) — Array Sum

Summing all elements of an array: $s = \sum_{i=1}^N a_i$

FLOPs: N additions

Memory: Read $N \times 8$ bytes (each element read once)

Understanding Performance

Practice: Solutions (2/3) — Array Sum

Summing all elements of an array: $s = \sum_{i=1}^N a_i$

FLOPs: N additions

Memory: Read $N \times 8$ bytes (each element read once)

Arithmetic intensity: $N/(8N) = 0.125$ FLOPs/byte $\rightarrow$ **Memory-bound**

Implication: Optimising the arithmetic (e.g. with Numba) gives little gain here — the bottleneck is memory bandwidth, not compute speed. NumPy's vectorised sum is already near the memory bandwidth limit.



Understanding Performance

Practice: Solutions (3/3) — Pairwise Distances

Computing all pairwise Euclidean distances between N points in d dimensions (e.g. k -NN, clustering, similarity search)



Understanding Performance

Practice: Solutions (3/3) — Pairwise Distances

Computing all pairwise Euclidean distances between N points in d dimensions (e.g. k -NN, clustering, similarity search)

FLOPs: N^2 pairs $\times$ $2d$ operations per pair $= 2dN^2$ FLOPs

- Per pair: d subtractions, d multiply-adds (squared differences), 1 sum

Memory: Read $N \times d \times 8$ bytes — each point reused N times



Understanding Performance

Practice: Solutions (3/3) — Pairwise Distances

Computing all pairwise Euclidean distances between N points in d dimensions (e.g. k -NN, clustering, similarity search)

FLOPs: N^2 pairs $\times$ $2d$ operations per pair $= 2dN^2$ FLOPs

► Per pair: d subtractions, d multiply-adds (squared differences), 1 sum

Memory: Read $N \times d \times 8$ bytes — each point reused N times

Arithmetic intensity: $\frac{2dN^2}{8Nd} = \frac{N}{4}$ FLOPs/byte — grows with dataset size!

Understanding Performance

Practice: Solutions (3/3) — Pairwise Distances

Computing all pairwise Euclidean distances between N points in d dimensions (e.g. k -NN, clustering, similarity search)

FLOPs: N^2 pairs $\times$ $2d$ operations per pair $= 2dN^2$ FLOPs

- Per pair: d subtractions, d multiply-adds (squared differences), 1 sum

Memory: Read $N \times d \times 8$ bytes — each point reused N times

Arithmetic intensity: $\frac{2dN^2}{8Nd} = \frac{N}{4}$ FLOPs/byte — grows with dataset size!

Note: d cancels out — intensity depends only on N , not the number of features!

- This holds as long as the full dataset ($N \times d \times 8$ bytes) fits in last-level cache. Once it overflows, points must be re-fetched from DRAM and intensity drops — the problem becomes partially memory-bound.

Use case	N	d	Intensity	
Tabular data (e.g. clinical kNN)	200	10	50 FLOPs/byte	Compute-bound
Image/word embeddings	1 000	64	250 FLOPs/byte	Strongly compute-bound
Large-scale similarity search	50 000	32	12 500 FLOPs/byte	Extremely compute-bound

Numba and GPU excel here — this is why GPU-accelerated kNN is orders of magnitude faster.

Understanding Performance

Summary

What you should remember:

1. Algorithmic Intensity = FLOPs / Bytes

- ▶ Tells you if code is compute- or memory-bound
- ▶ Calculate it for your algorithms!

2. Low intensity (< 1): Memory-bound

- ▶ Focus: Cache optimization, memory layout, reduce transfers
- ▶ NumPy helps, Numba helps less, GPU might not help

3. High intensity (> 10): Compute-bound

- ▶ Focus: JIT compilation, parallelization, GPU
- ▶ Numba helps a lot, GPU extremely effective

4. Mandelbrot is highly compute-bound (≈ 45 FLOPs/byte)

- ▶ Perfect candidate for Numba and GPU
- ▶ This is why we chose it for the course!

Next: Let's profile to confirm our analysis, then apply Numba!



10-minute break



Studio Session

Mini-Project 1

Requirements & Deadline

Today we work towards finishing the first mini-project submission.

Due: Tuesday 3 March 23:59 — everything you have built so far counts toward MP1.

Submit the following to Moodle:

1. **Code:** Naive, NumPy, Numba (M3) + float32 vs float64 experiments (M4)
One or more `.py` or `.ipynb` files
2. **Performance notebook:** today's 4 milestones — profiling analysis (M1–M2), timing/speedup results (M3), data type comparison (M4), optional extension (M5*)
Jupyter Notebook (`.ipynb`) or exported PDF (from Word, $\text{\LaTeX}$, etc.)
3. **Reflection** (150+ words): biggest challenge, most surprising result, what you learned
Copy-paste text directly in the Moodle submission text field
4. **Git log:**
`git log --format="%ad %h %s" --date=short --stat > git_log.txt`

Studio Session

Milestones Overview

Work through milestones at your own pace. Each ends with a Git commit.

M1 Function-level profiling — cProfile on naive and NumPy

M2 Line-level profiling — line_profiler, create profiling report

M3 Numba implementation — implement @njit, benchmark all versions

M4 Data type optimization — float32 vs float64 comparison

M5* Extension: Parallel Numba — prange and parallel=True

Slides that follow contain instructions for each milestone.

Reference material (Numba concepts, benchmarking methodology) is included after the milestone slides — use as needed.

Deep Profiling

Why Profile?

"I know where the bottleneck is."

Studies show programmers guess wrong >80% of the time.

`time.perf_counter()`

- ✓ Measures total wall-clock time
- ✓ Tells you *how slow*
- ✗ Doesn't tell you *why*
- ✗ Can't pinpoint the line

`cProfile + line_profiler`

- ✓ Shows time per function
- ✓ Shows time per *line*
- ✓ Counts how often each is called
- ✓ Removes guesswork entirely

	naive	NumPy
mandelbrot_point called	262 144×	0×
Time in inner loop	95%	—

cProfile output like this tells you exactly where to focus.

Deep Profiling

Milestone 1: Function-Level Profiling

Use cProfile on both versions:

```
import cProfile, pstats
from mandelbrot import mandelbrot_naive, mandelbrot_numpy

cProfile.run('mandelbrot_naive(-2, 1, -1.5, 1.5, 512, 512)',
             'naive_profile.prof')
cProfile.run('mandelbrot_numpy(-2, 1, -1.5, 1.5, 512, 512)',
             'numpy_profile.prof')

for name in ('naive_profile.prof', 'numpy_profile.prof'):
    stats = pstats.Stats(name)
    stats.sort_stats('cumulative')
    stats.print_stats(10)
```

Questions to answer:

- ▶ Which function takes most total time?
- ▶ Are there functions called surprisingly many times?
- ▶ How does the NumPy profile compare to naive?

✓ **Done?** Add the cProfile output for both versions and your answers to the questions above to your mini-report → commit

Deep Profiling

Milestone 1: What to Look For in cProfile Output

ncalls	tottime	cumtime	function
1	0.000	15.342	mandelbrot_naive
262144	12.456	12.456	mandelbrot_point
26214400	2.886	2.886	built-in abs

Key columns:

- ▶ **ncalls:** How many times function called
- ▶ **tottime:** Time in this function only (excluding sub-calls)
- ▶ **cumtime:** Total time including sub-calls
- ▶ **function:** Function name

Questions to answer:

1. Which function takes most total time?
2. Are there functions called surprisingly many times?
3. How does NumPy profile compare to naive?
4. Where does NumPy spend its time?

Document your findings! Add a summary table to your mini-report.

Deep Profiling

Milestone 2: Line-Level Profiling

Add @profile decorator and run line_profiler:

```
@profile # Add this decorator
def mandelbrot_naive(xmin, xmax, ymin, ymax, width, height, max_iter=100):
    x = np.linspace(xmin, xmax, width)
    y = np.linspace(ymin, ymax, height)
    result = np.zeros((height, width), dtype=int)
    for i in range(height):
        for j in range(width):
            c = x[j] + 1j * y[i]
            z = 0
            for n in range(max_iter):
                if abs(z) > 2:
                    result[i, j] = n
                    break
                z = z*z + c
            else:
                result[i, j] = max_iter
    return result
```

```
kernprof -l -v mandelbrot_script.py # or use Spyder: Run -> Profile (F10)
```

Key finding: The `abs(z) > 2` check and `z*z+c` take 50–60% of total time.

✓ **Done?** Commit → continue to profiling report

Deep Profiling

Milestone 2: cProfile vs line_profiler

	cProfile	line_profiler
Granularity	Function-level	Line-by-line
Overhead	Low ($\sim 5\text{--}20\%$)	High (10–100× slower)
Use for	Finding slow functions	Pinpointing the exact line
Safe to benchmark?	Yes (rough timings)	No

Why is line_profiler so slow?

- ▶ Each line is timed individually using Python's low-level tracing hooks
- ▶ A line inside the Mandelbrot inner loop may execute $>26\,000\,000$ times — each gets its own timestamp call
- ▶ This overhead dominates the actual computation time

Rule: Use line_profiler to find *which line* is the bottleneck. Once you know, **remove** @profile and benchmark with plain perf_counter.

Deep Profiling

Milestone 2: Add Profiling Section to Mini-Report

Add a profiling section to your MP1 mini-report. In your own words, answer:

1. **cProfile on naive vs NumPy:** How many functions appear in each profile? What does this difference tell you about where the work actually happens?
2. **line_profiler on naive:** Which lines dominate runtime? What fraction of total time is spent in the inner loop?
3. Based on your profiling results: why is NumPy faster than naive Python?
4. What would you need to change to make the naive version faster? (hint: what does line_profiler tell you about the inner loop?)

Support your answers with numbers from your profiling output — copy-pasting terminal output directly into your mini-report is fine.

✓ **Done?** Commit → move on to Milestone 3 (Numba)

Numba JIT

What is Numba?

- ▶ **Numba = JIT (Just-In-Time) compiler for Python**
 - ▶ Compiles Python functions to machine code at runtime via LLVM
 - ▶ Can approach C/Fortran speeds for numerical loops
- ▶ **Why Python is slow — and how Numba fixes it:**
 - ▶ Python checks the type of every variable at every operation (dynamic typing) — this overhead dominates in tight loops
 - ▶ On the **first call**, Numba **infers concrete types** from the actual argument types (e.g. `float64`, `int32`) and compiles a type-specialised version via LLVM — just like a C compiler would
 - ▶ Subsequent calls execute the compiled machine code directly, bypassing the interpreter entirely
 - ▶ **Consequence:** the first call is slow (compilation happens then) — always warm up before timing!
- ▶ **When Numba helps:**
 - ▶ **Good:** Numerical loops, array operations, math functions called many times
 - ▶ **Bad:** String operations, file I/O, object manipulation, one-off calls
- ▶ **Perfect for Mandelbrot:** pure numerical computation, tight inner loops, 1M+ calls

Numba JIT

Two Main Decorators

@jit — Basic JIT

```
from numba import jit

@jit
def mandelbrot_point(c, max_iter=100):
    z = 0j
    for n in range(max_iter):
        if z.real*z.real + z.imag*z.imag >
            4.0:
            return n
        z = z*z + c
    return max_iter
```

- Falls back to Python if compilation fails
- Safer, but potentially slower

@njit — Nopython mode (use this!)

```
from numba import njit

@njit
def mandelbrot_point(c, max_iter=100):
    z = 0j
    for n in range(max_iter):
        if z.real*z.real + z.imag*z.imag >
            4.0:
            return n
        z = z*z + c
    return max_iter
```

- No Python fallback — fails explicitly
- Fastest performance

Note: $z.\text{real}*z.\text{real} + z.\text{imag}*z.\text{imag} > 4.0$ replaces $\text{abs}(z) > 2$ — see next slide for why.

Numba JIT

Why Rewrite $\text{abs}(z) > 2$?

Until now you have written the escape test as:

```
if abs(z) > 2:    # naive version
```

In Numba we write instead:

```
if z.real*z.real + z.imag*z.imag > 4.0:    # Numba version
```

Quick reminder — complex numbers:

- ▶ A complex number $z = a + bi$, where a = real part, b = imaginary part, $i = \sqrt{-1}$
 - ▶ Magnitude: $|z| = \sqrt{a^2 + b^2}$, so $|z|^2 = a^2 + b^2$ (no square root needed!)
 - ▶ Therefore: $|z| > 2 \Leftrightarrow a^2 + b^2 > 4$, i.e. $z.\text{real}*z.\text{real} + z.\text{imag}*z.\text{imag} > 4.0$

Two reasons to prefer the explicit form:

1. **Avoids a square root.** The comparison is mathematically equivalent but `sqrt` is expensive — skipping it saves FLOPs on every iteration.
2. **Numba handles `abs()` on complex numbers via Python's object system**, which adds overhead. Accessing `.real` and `.imag` directly stays in compiled code.

This is the kind of micro-optimisation that profiling helps you find and justify.

Numba JIT

Common Numba Gotchas

► **Issue 1:** First call is slow — don't include in benchmark!

```
_ = mandelbrot_numba(-2, 1, -1.5, 1.5, 64, 64) # warm-up
start = time.perf_counter()
result = mandelbrot_numba(-2, 1, -1.5, 1.5, 1024, 1024) # now time this
```

► **Issue 2:** Not all Python features supported

- **Can't use:** Python lists, dicts, classes
- **Can use:** NumPy arrays, basic numeric types, math functions
- $z = 0j$ not $z = 0$ — Numba needs a complex literal for type inference

► **Issue 3:** Type inference errors — can specify explicitly:

```
from numba import njit, int32, complex128

@njit
def mandelbrot_point(c: complex128, max_iter: int32) -> int32:
    ...
```

Numba JIT

Two Approaches: Hybrid vs. Fully Compiled

Approach A — Hybrid

```
@njit
def mandelbrot_point_numba(c, max_iter=100):
    z = 0j
    for n in range(max_iter):
        if z.real*z.real + z.imag*z.imag >
            4.0:
            return n
        z = z*z + c
    return max_iter

def mandelbrot_hybrid(...):
    # outer loops still in Python
    for i in range(height):
        for j in range(width):
            c = x[j] + 1j * y[i]
            result[i,j] =
                mandelbrot_point_numba(c, max_iter)
```

Problem: outer loops still Python → 1M call overhead
 If `mandelbrot_hybrid` also gets `@njit`, Numba inlines
 the inner call and the result is equivalent to B

Approach B — Fully compiled (recommended)

```
@njit
def mandelbrot_naive_numba(...):
    x = np.linspace(xmin, xmax, width)
    y = np.linspace(ymin, ymax, height)
    result = np.zeros((height, width),
                      dtype=np.int32)

    for i in range(height):
        for j in range(width):
            c = x[j] + 1j * y[i]
            z = 0j
            n = 0
            while n < max_iter and \
                z.real*z.real+z.imag*z.imag <=
                4.0:
                z = z*z + c; n += 1
            result[i, j] = n
    return result
```

All loops compile to machine code
 Equivalent to A+`@njit`, but explicit in one function
 — easier to reason about

Numba JIT

Fully Compiled Naive: All Loops in Machine Code

Approach B — Numba-naive: decorate the entire function:

```
from numba import njit
import numpy as np

@njit                                     # <-- entire function compiled!
def mandelbrot_naive_numba(xmin, xmax, ymin, ymax,
                           width, height, max_iter=100):
    """Fully JIT-compiled Mandelbrot --- structure identical to naive."""
    x = np.linspace(xmin, xmax, width)
    y = np.linspace(ymin, ymax, height)
    result = np.zeros((height, width), dtype=np.int32)

    for i in range(height):              # compiled loop
        for j in range(width):           # compiled loop
            c = x[j] + 1j * y[i]
            z = 0j                        # complex literal: type inference works!
            n = 0
            while n < max_iter and (z.real*z.real + z.imag*z.imag) <= 4.0:
                z = z*z + c
                n += 1
            result[i, j] = n
    return result

# Warm-up (triggers compilation --- don't time this!)
_ = mandelbrot_naive_numba(-2, 1, -1.5, 1.5, 64, 64)
```

Numba JIT

Milestone 3: Implement & Compare Approaches

Your Task: Implement both approaches and measure the difference.

```
import time, statistics

def bench(fn, *args, runs=5):
    fn(*args) # extra warm-up
    times = []
    for _ in range(runs):
        t0 = time.perf_counter()
        fn(*args)
        times.append(time.perf_counter() - t0)
    return statistics.median(times)

# Warm up (triggers JIT compilation -- exclude from timing)
_ = mandelbrot_hybrid(-2, 1, -1.5, 1.5, 64, 64)
_ = mandelbrot_naive_numba(-2, 1, -1.5, 1.5, 64, 64)

t_hybrid = bench(mandelbrot_hybrid, -2, 1, -1.5, 1.5, 1024, 1024)
t_full = bench(mandelbrot_naive_numba, -2, 1, -1.5, 1.5, 1024, 1024)

print(f"Hybrid: {t_hybrid:.3f}s")
print(f"Fully compiled: {t_full:.3f}s")
print(f"Ratio: {t_hybrid/t_full:.1f}x")
```

Expected: fully compiled is usually faster — outer-loop dispatch to JIT adds up over 1M pixels.

✓ **Done?** Commit → log in Performance Tracker

Numba JIT

Milestone 3: Benchmark All Versions

Measure full speedup chain:

```
import time, statistics

def bench(fn, *args, runs=5):
    fn(*args) # warm-up
    times = []
    for _ in range(runs):
        t0 = time.perf_counter()
        fn(*args)
        times.append(time.perf_counter() - t0)
    return statistics.median(times)

width, height = 1024, 1024
args = (-2, 1, -1.5, 1.5, width, height)

t_naive = bench(mandelbrot_naive, *args)
t_numpy = bench(mandelbrot_numpy, *args)
t_numba = bench(mandelbrot_naive_numba, *args)

print(f"Naive: {t_naive:.3f}s")
print(f"NumPy: {t_numpy:.3f}s ({t_naive/t_numpy:.1f}x)")
print(f"Numba: {t_numba:.3f}s ({t_naive/t_numba:.1f}x)")
```

✓ **Done?** Add timings and speedup table (naive / NumPy / Numba) to your mini-report → log in Performance Tracker → commit

Numba JIT

Milestone 3: Expected Results

Typical speedups (1024×1024 grid):

Version	Typical range	M1 Max (ref.)	Speedup vs. Naive
Naive Python	20–60s	3.66s	$1\times$ (baseline)
NumPy	1–5s	0.71s	$5\text{--}50\times$ (M1: $5\times$)
Numba	0.05–0.5s	0.073s	$50\text{--}200\times$ (M1: $\sim 49\times$)

Your numbers will differ! Modern hardware (M1/M2/M3, recent Intel/AMD) runs naive Python faster than older CPUs — which *compresses* all speedup ratios. As seen in our class performance tracker, naive runtimes range from ~ 1 s to ~ 40 s depending on hardware generation.

Why is Numba faster than NumPy here?

- ▶ NumPy must evaluate all `max_iter` iterations for every pixel, then mask
- ▶ Numba compiles the early-exit: points that escape skip remaining iterations
- ▶ This is a *structural* advantage — not just compiled code
- ▶ Numba also eliminates Python loop overhead and array allocation per step

Key insight: Numba beats NumPy by $10\times$ on M1 — not because NumPy is slow, but because the Mandelbrot iteration structure *favours loops with early exit*.

Data Type Optimization

Precision vs. Performance

► Floating-point types in NumPy:

Type	Bytes	Precision	Range
float16	2	3 decimals	$\pm 65,504$
float32	4	7 decimals	$\pm 3.4 \times 10^{38}$
float64	8	15 decimals	$\pm 1.8 \times 10^{308}$

► Tradeoffs:

- **float64 (default):** Best precision, slowest, most memory
- **float32:** Half the memory, often 2× faster, usually sufficient
- **float16:** Quarter the memory, very limited range/precision

► When precision matters:

- Iterative algorithms (errors accumulate)
- Subtracting similar numbers (catastrophic cancellation)
- Very large or very small numbers

► When float32 is fine:

- Graphics, visualization
- Machine learning (most models)
- Mandelbrot set (7 decimals plenty!)

Data Type Optimization

Milestone 4: Test Different Precisions

Modify your Numba version to accept a dtype parameter:

```
from numba import njit
import numpy as np, time

@njit
def mandelbrot_numba_typed(xmin, xmax, ymin, ymax,
                           width, height, max_iter=100, dtype=np.float64):
    x = np.linspace(xmin, xmax, width).astype(dtype)
    y = np.linspace(ymin, ymax, height).astype(dtype)
    result = np.zeros((height, width), dtype=np.int32)
    for i in range(height):
        for j in range(width):
            c = x[j] + 1j * y[i]
            result[i, j] = mandelbrot_point_numba(c, max_iter)
    return result

for dtype in [np.float16, np.float32, np.float64]:
    t0 = time.perf_counter()
    mandelbrot_numba_typed(-2, 1, -1.5, 1.5, 1024, 1024, dtype=dtype)
    print(f"{dtype.__name__}: {time.perf_counter()-t0:.3f}s")
```

Data Type Optimization

Milestone 4: Visual Comparison

Compare accuracy and generate side-by-side images:

```
import matplotlib.pyplot as plt
import numpy as np

r16 = mandelbrot_numba_typed(-2, 1, -1.5, 1.5, 1024, 1024, dtype=np.float16)
r32 = mandelbrot_numba_typed(-2, 1, -1.5, 1.5, 1024, 1024, dtype=np.float32)
r64 = mandelbrot_numba_typed(-2, 1, -1.5, 1.5, 1024, 1024, dtype=np.float64)

fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for ax, result, title in zip(axes, [r16, r32, r64],
                             ['float16', 'float32', 'float64 (ref)']):
    ax.imshow(result, cmap='hot')
    ax.set_title(title); ax.axis('off')
plt.savefig('precision_comparison.png', dpi=150)

print(f"Max diff float32 vs float64: {np.abs(r32-r64).max()}")
print(f"Max diff float16 vs float64: {np.abs(r16-r64).max()}")
```



Data Type Optimization

Milestone 4: What to Look For

Run your experiments and observe:

- ▶ **Speed:** Does float32 actually run faster than float64 on your hardware? By how much? Is float16 faster or slower — and why might that surprise you?
- ▶ **Visual quality:** Zoom in on a detailed region of the Mandelbrot set. Can you see artefacts with float16? What about float32?
- ▶ **Recommendation:** Based on what you observe, which precision would you choose for production use, and why?

Hint: The answer is not always obvious — modern CPUs handle different precisions differently, and the memory bandwidth argument does not always dominate.

Important for later: The choice of precision matters much more on GPUs (Week 10).

Data Type Optimization

Milestone 4: Add Data Type Section to Mini-Report

Add a data type section to your MP1 mini-report. Include:

- ▶ Your measured runtimes for float16, float32, float64
- ▶ Side-by-side images (or a zoomed crop showing where artefacts appear)
- ▶ A brief recommendation: which precision would you use and why?

No specific format required — a few sentences and your images is enough. The goal is to show you understand the precision/performance trade-off.

✓ **Done?** Add and commit work and results.

Numba JIT

Milestone 5* (Optional extension): Parallel Numba

Try different Numba optimizations:

Experiment 1: @jit vs. @njit

- ▶ Replace @njit with @jit
- ▶ Measure performance difference
- ▶ Usually @njit is faster

Experiment 2: Parallel Numba (advanced)

```
from numba import njit, prange

@njit(parallel=True)
def mandelbrot_numba_parallel(...):
    # ...
    for i in prange(height): # Parallel!
        for j in range(width):
            # ...
```

- ▶ Try parallel=True and prange
- ▶ May give 2-4× speedup on multi-core

Experiment 3: Scaling with grid size

- ▶ Test 512^2 , 1024^2 , 4096^2 (or larger)
- ▶ Plot grid size vs. time for all implementations
- ▶ Does speedup scale with problem size?

Memory warning:

- ▶ Memory scales as $N^2 \times d$ bytes
($d = 8$ for float64, 4 for float32):

N	per array (float64)
1 024	~8 MB
4 096	~128 MB
8 192	~512 MB

- ▶ NumPy holds several arrays at once
(C, Z, result, masks) — multiply by 3–5×

Exceeding RAM causes swapping + crashes!



Wrap-up

15:00

Wrap-up

Reproducible Benchmarking – A Professional Skill

- ▶ **Problem:** “I got a $3\times$ speedup” – but can anyone reproduce it?
 - ▶ Did you include warm-up runs? First call to Numba compiles kernels
 - ▶ Did you run multiple trials? Single measurements have noise
 - ▶ Did you record the hardware? Results differ between machines
- ▶ **Professional benchmarking methodology:**
 1. **Warm-up:** Run function once before timing (JIT compilation, cache warm-up)
 2. **Multiple trials:** Run 3–5 times, report median and std deviation
 3. **Proper timing:** Use `time.perf_counter()`, not `time.time()`
 4. **Record environment:** Python version, hardware, OS, library versions
- ▶ **In your mini-report, always include:**
 - ▶ Hardware specs (CPU model, cores, RAM, GPU if used)
 - ▶ Library versions (`numpy.__version__`, etc.)
 - ▶ Measurement methodology (warm-up runs, number of trials)
 - ▶ Results with variance: $\mu \pm \sigma$, not just a single number
- ▶ **Why this matters:** Industry and research both require reproducible results. In industry, a benchmark guides hardware purchases. In research, it validates your method.

Wrap-up

What We Accomplished Today

► Profiling mastery:

- Systematic profiling methodology
- cProfile and line_profiler usage
- Identified bottlenecks quantitatively

► Numba optimization:

- Applied JIT compilation with `@njit`
- Achieved 50–200 $\times$ speedup over naive Python (platform-dependent)
- Understood when Numba helps vs. doesn't

► Data type optimization:

- Explored float16, float32, float64 tradeoffs
- Measured performance vs. precision
- Made informed decisions about data types

► MP1 foundation complete!

- Three working implementations
- Comprehensive performance analysis
- Ready to finalize MP1



Wrap-up

MP1 Checklist — Due Tuesday 3 March 23:59

Before you submit to Moodle, tick off:

- ☐ Naive, NumPy, and Numba implementations working and tested
- ☐ float32 vs float64 experiment with visual and timing comparison
- ☐ Performance notebook (.ipynb or PDF): speedup results + profiling analysis
- ☐ Reflection written in Moodle submission text field (150+ words)
- ☐ Git log exported: `git log --format="%ad %h %s" --date=short --stat > git_log.txt`

Bundle code, mini-report, and git log into a single zip and upload to Moodle.

Before Next Week

Complete at Home

Must complete (if not done today):

1. Finalize MP1 submission and submit in Moodle

Prepare for Week 4: Parallel Computing

- ▶ Listen to the Week 4 podcast
- ▶ Watch the Week 4 video
- ▶ Read the assigned sections in the textbook

Next week (Week 4):

- ▶ Multiple CPU cores → another significant speedup!
- ▶ MP1 peer review — bring your laptop and be ready to discuss your work

Questions?

Questions?

Contact:

Jimmy Jessen Nielsen
jjn@es.aau.dk

Complete MP1 by Tuesday 3 March 23:59!
Next week: Parallelization begins!

You are welcome to leave and continue at home,
or stay and keep working — help is available until 16:15.